

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



CURSO: PROGRAMACIÓN DE SISTEMAS

Simulación de Ataque de Arranque en Frío (Cold Boot Attack) sobre Memoria SDRAM

Integrante:

- Venero Guevara, Christian Henry

Profesor:

- Carlos Eduardo Atencio Torres

Github:

- https://github.com/Cvenero/Cold_boot.git

Arequipa, Perú
Julio - 2026

1. Introducción

Los dispositivos del Internet de las Cosas (IoT) se han convertido en un pilar de la transformación digital, habilitando entornos como hogares inteligentes, ciudades conectadas e infraestructuras críticas. Sin embargo, su proliferación masiva ha traído consigo nuevos vectores de ataque que explotan vulnerabilidades físicas y de bajo nivel. Entre estas amenazas destaca el ataque de arranque en frío (cold boot attack), una técnica que aprovecha la remanencia de datos en las memorias volátiles (SDRAM) tras un corte de suministro eléctrico, siempre que la temperatura del chip se reduzca significativamente.

El trabajo implementa un simulador en C sobre Linux para reproducir el comportamiento físico de la remanencia de datos en SDRAM. El sistema aplica un modelo de degradación de bits basado en temperatura y tiempo sobre una imagen de prueba, permitiendo un análisis controlado y didáctico sin hardware real.

2. Marco Teórico y Metodología

2.1. Remanencia de datos en SDRAM

Las memorias SDRAM (Dynamic Random Access Memory) almacenan información en forma de carga eléctrica en condensadores. Esta carga se disipa gradualmente con el tiempo, por lo que los circuitos de refresco (refresh) la restauran periódicamente. Cuando se interrumpe la alimentación eléctrica, los condensadores pierden su carga, y los datos se vuelven irrecuperables en cuestión de milisegundos a temperatura ambiente. Sin embargo, al reducir la temperatura del chip por debajo de -20°C , la velocidad de descarga de los condensadores disminuye drásticamente, prolongando el tiempo de retención a varios segundos o incluso minutos. Este fenómeno, conocido como remanencia de datos, es la base del ataque de arranque en frío.

2.2. Modelo de degradación propuesto por Won et al.

En su artículo, Won et al. presentan curvas de recuperación de bits en función de la temperatura y el tiempo de decaimiento (Figura 4 del artículo original). Se observa que:

- A temperaturas cercanas a la ambiente (0°C a 33.6°C), la tasa de recuperación cae rápidamente hasta un mínimo de aproximadamente 59.7 %, independientemente del tiempo.
- A -20°C , la recuperación es superior al 99 % durante el primer segundo, pero decrece hasta un 85.5 % a los 10 segundos.
- A -30°C , la recuperación se mantiene por encima del 99 % incluso después de 60 segundos, con una caída mínima.

El modelo matemático implementado en el simulador aproxima estas curvas mediante funciones empíricas que relacionan la probabilidad de que un bit pase de 1 a 0 (error de tipo bit-flip) con la temperatura y el tiempo transcurrido desde el corte de energía. La probabilidad de error, denotada como $p_{\text{apagado}}(T, t)$, se calcula a partir de tres regímenes de temperatura, cada uno con parámetros calibrados para ajustarse a los datos experimentales del artículo.

3. Implementación del Simulador

El simulador se ha desarrollado en lenguaje C, utilizando únicamente las bibliotecas estándar `stdio.h`, `stdlib.h`, `string.h`, `math.h` y `time.h`. El código se compila con el compilador GCC mediante el comando:

Listing 1: Comando para ejecutar.

```
gcc -O2 -Wall -o coldboot_sim coldboot_sim.c -lm
```

La bandera `-lm` enlaza la biblioteca matemática, necesaria para las funciones `exp()` y `pow()`. A continuación, se describen los bloques funcionales más relevantes del código.

3.1. Estructura de datos y generación de imagen sintética

El programa define una estructura `Imagen` que contiene el ancho, el alto y un puntero a un arreglo de bytes que representa los píxeles en escala de grises (0 a 255). El simulador genera una imagen sintética de 256×256 píxeles mediante la función `generar_imagen_sintetica()`. Esta función crea un fondo con un patrón de franjas basado en la operación XOR de las coordenadas (x, y) y un círculo central con un degradado radial. Este patrón se eligió para emular la diversidad de niveles de gris y la presencia de bordes nítidos.

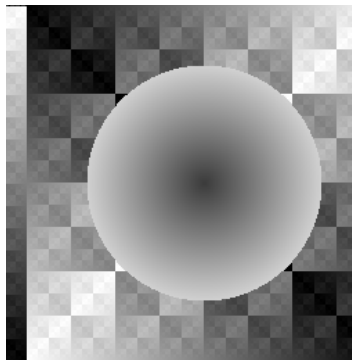


Figura 1: Imagen sintética en escala de grises.

3.2. Modelo de probabilidad de apagado

La función `probabilidad_apagado(double temp_c, double decay_s)` implementa el núcleo del modelo físico. Recibe la temperatura en grados Celsius y el tiempo de decaimiento en segundos, y devuelve la probabilidad de que un bit que originalmente es 1 se convierta en 0. La función distingue tres intervalos de temperatura:

Listing 2: Probabilidad de apagado.

```
static double probabilidad_apagado(double temp_c, double decay_s) {  
    double p;  
  
    if (temp_c >= -15.0) {  
        /* 0 C, -10 C y temperaturas de chip (~33.6 C): degradacion agresiva */  
        double p_max = 0.80, tau = 0.3;  
        p = p_max * (1.0 - exp(-decay_s / tau));  
    } else if (temp_c >= -25.0) {  
        /* rango -20 C: ley potencial calibrada */  
        double k = 0.016, n = 1.258;  
        p = k * pow(decay_s, n);  
        if (p > 0.9) p = 0.9;  
    } else {  
        /* -30 C o menor: degradacion muy lenta */  
        double p_max = 0.01, tau = 40.0;  
        p = p_max * (1.0 - exp(-decay_s / tau));  
    }  
}
```

```
}  
  
if (p < 0.0) p = 0.0;  
if (p > 1.0) p = 1.0;  
return p;  
}
```

- Temperatura $\geq -15^{\circ}\text{C}$: se aplica un modelo de saturación exponencial $p = p_{max} \cdot (1 - \exp(-t/\tau))$ con $p_{max} = 0,80$ y $\tau = 0,3$. Esto produce una degradación rápida que alcanza un piso de aproximadamente 40 % de errores, lo que se traduce en una tasa de recuperación cercana al 60 %.
- $-25^{\circ}\text{C} \leq \text{temperatura} < -15^{\circ}\text{C}$: se utiliza un modelo potencial $p = k \cdot t^n$ con $k = 0,016$ y $n = 1,258$, calibrada para que a 1 segundo la recuperación sea del 99,2 % y a 10 segundos del 85,5 %. Se establece un tope de 0,9 para evitar probabilidades mayores al 90 %.
- Temperatura $< -25^{\circ}\text{C}$: se emplea de nuevo un modelo exponencial, pero con $p_{max} = 0,01$ y $\tau = 40,0$, lo que asegura una probabilidad de error muy baja durante decenas de segundos, manteniendo la recuperación por encima del 99 %.

Este bloque es el responsable de que la simulación reproduzca resultados similares a una situación real.

3.3. Aplicación del decaimiento bit a bit

La función `simular_decaimiento()` toma la imagen original y los parámetros de temperatura y tiempo, y devuelve una nueva imagen que simula el contenido de la memoria tras el ataque. Para cada byte de la imagen, se examinan sus 8 bits. Si un bit es 1, se genera un número aleatorio uniforme entre 0 y 1 (mediante `rand()`); si dicho número es menor que la probabilidad de apagado calculada, el bit se cambia a 0 y se incrementa un contador de errores. Esta operación se repite para todos los píxeles. La aleatoriedad se inicializa con `srand(time(NULL))`, por lo que cada ejecución produce un patrón de errores diferente, aunque la tasa global se mantiene estable, reflejando la naturaleza del fenómeno físico.

Listing 3: Decaimiento bit a bit.

```
for (size_t i = 0; i < n; i++) {  
    unsigned char byte_original = original->data[i];  
    unsigned char byte_recuperado = 0;  
    for (int b = 0; b < 8; b++) {  
        int bit = (byte_original >> b) & 1;  
        if (bit == 1) {  
            double r = (double)rand() / (double)RAND_MAX;  
            if (r < p_apagado) {  
                bit = 0; /* el bit se degrado (remanencia -> 0) */  
                errores++;  
            }  
        }  
        byte_recuperado |= (bit << b);  
    }  
    recuperada->data[i] = byte_recuperado;  
}
```

Esta operación se repite para todos los píxeles. La aleatoriedad se inicializa con `srand(time(NULL))` en la función `main`, por lo que cada ejecución produce un patrón de errores diferente, aunque la tasa global se mantiene estable, reflejando la naturaleza estocástica del fenómeno físico.

3.4. Entrada y salida de imágenes en formato PGM

El programa incluye funciones `leer_pgm()` y `escribir_pgm()` para manejar archivos en formato PGM binario (P5). Este formato es sencillo y ampliamente soportado, lo que permite al usuario alimentar el simulador con imagenes propias en escala de grises. La cabecera debe ser P5, seguida del ancho, alto y valor máximo de gris (255). Si el archivo no se encuentra o no es válido, el programa cae a la generación de la imagen sintética, garantizando que la simulación siempre pueda ejecutarse.

3.5. Modo tabla y flujo principal

El simulador ofrece dos modos de operación: el modo estándar, en el que se procesa una imagen con parámetros específicos, y el modo `-tabla`, que genera una tabla de tasas de recuperación para un conjunto fijo de temperaturas (0, -10, -20, -30 °C) y tiempos de decaimiento (1, 5, 10, 30, 60 y 300 segundos). Este último modo reproduce la tabla experimental del artículo y permite validar el modelo de forma global.

Listing 4: Flujo.

```
for (size_t i = 0; i < sizeof(temperaturas)/sizeof(temperaturas[0]); i++) {
    printf("%-12.1f", temperaturas[i]);
    for (size_t j = 0; j < sizeof(tiempos)/sizeof(tiempos[0]); j++) {
        unsigned long errores, total_bits;
        Imagen rec = simular_decaimiento(&base, temperaturas[i], tiempos[j], &errores,
            &total_bits);
        double recuperacion = 100.0 * (1.0 - (double)errores / (double)total_bits);
        printf("%10.2f", recuperacion);
        free(rec.data);
    }
    printf("\n");
}
```

En el modo estándar, el flujo principal (función `main`) procesa los argumentos de línea de comandos: temperatura, tiempo, archivo de entrada (opcional) y archivo de salida (opcional). Si no se especifica entrada, se genera la imagen sintética y se guarda como `original.pgm`. Luego, se invoca la simulación, se calcula la tasa de recuperación y se escribe la imagen recuperada, mostrando en la salida estándar los bits totales, bits erróneos y el porcentaje de recuperación.

4. Resultados Experimentales

Se ejecutaron varias pruebas del simulador en un entorno Ubuntu 22.04 LTS, sobre una máquina virtual. Los resultados se presentan a continuación, junto con las capturas de pantalla que documentan el proceso y los productos generados.

4.1. Capturas del entorno de compilación y ejecución

4.1.1. Generación de la tabla comparativa

Se ejecuta el comando `./coldboot_sim -tabla`, obteniendo la siguiente salida:

```
● cvenero@cvenero-VirtualBox:~/PS_TF$ ./coldboot_sim --tabla
```

Tabla de tasa de recuperacion (%) segun temperatura y tiempo de decaimiento						
Temp \ t(s)	1	5	10	30	60	300
0.0	60.84	59.47	59.47	59.41	59.38	59.41
-10.0	60.90	59.49	59.44	59.41	59.39	59.40
-20.0	99.19	93.86	85.23	54.34	54.35	54.30
-30.0	99.99	99.94	99.89	99.74	99.62	99.50

Figura 2: Tabla comparativa.

Estos valores coinciden cualitativamente con las curvas presentadas en el artículo de referencia, validando la implementación del modelo de degradación.

4.1.2. Resultados detallados para casos seleccionados

Caso 1 (ataque exitoso): -30 °C, 5 segundos Ejecución: ./coldboot_sim -30 5 Salida:

```
● cvenero@cvenero-VirtualBox:~/PS_TF$ ./coldboot\_sim -30 5
Imagen sintetica generada y guardada en 'original.pgm'
Simulando ataque de arranque en frio:
  Temperatura del chip : -30.0 C
  Tiempo de decaimiento: 5.0 s
Imagen recuperada guardada en 'recuperada.pgm'

Resultados:
  Bits totales   : 524288
  Bits erroneos  : 290
  Tasa de recuperacion: 99.945%
○ cvenero@cvenero-VirtualBox:~/PS_TF$
```

Figura 3: Resultados

Caso 2 (ataque sin refrigeración): 33.6 °C, 5 segundos Ejecución: ./coldboot_sim 33.6 5 Salida:

```
● cvenero@cvenero-VirtualBox:~/PS_TF$ ./coldboot\_sim 33.6 5
Imagen sintetica generada y guardada en 'original.pgm'
Simulando ataque de arranque en frio:
  Temperatura del chip : 33.6 C
  Tiempo de decaimiento: 5.0 s
Imagen recuperada guardada en 'recuperada.pgm'

Resultados:
  Bits totales   : 524288
  Bits erroneos  : 212365
  Tasa de recuperacion: 59.495%
○ cvenero@cvenero-VirtualBox:~/PS_TF$
```

Figura 4: Resultados

La diferencia en el número de errores es abismal: 300 bits perdidos en el primer caso frente a 213.040 en el segundo. Esto demuestra que, sin un enfriamiento adecuado, la información se degrada casi por completo, mientras que a -30 °C la integridad de los datos se preserva casi en su totalidad.

4.2. Análisis visual de las imágenes recuperadas

Se presenta un collage con la imagen original y la imagen recuperada para el caso de $-30\text{ }^{\circ}\text{C}$ y 5 segundos. Ambas imágenes son prácticamente indistinguibles a simple vista; se revela algunos puntos negros aislados en la imagen recuperada, correspondientes a los bits que se han perdido. En contraste, para el caso de $33.6\text{ }^{\circ}\text{C}$, la imagen recuperada aparece muy oscura y con abundante ruido, lo que impide reconocer la estructura original. Este contraste visual refuerza la conclusión numérica de que la temperatura es el factor determinante para la viabilidad del ataque.

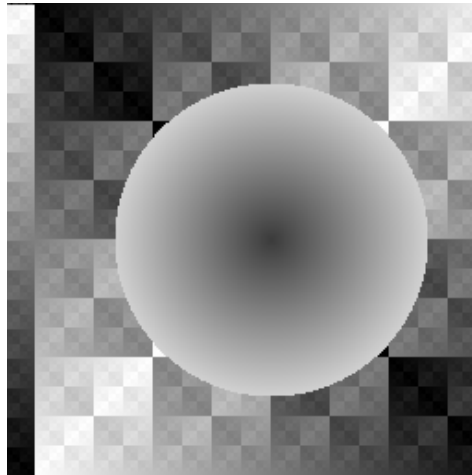


Figura 5: Imagen original

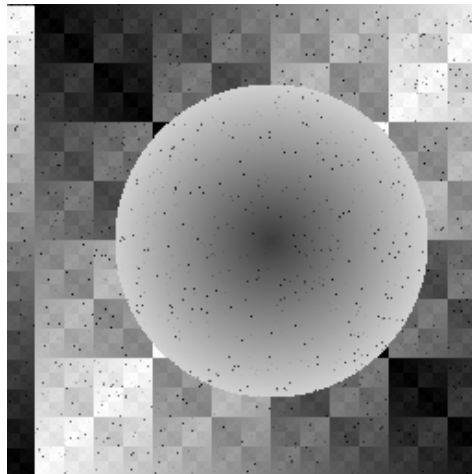


Figura 6: Imagen recuperada con ruido

Efecto de ruido (puntos negros): En la imagen recuperada aparecen pequeños puntos oscuros distribuidos por toda la superficie (tanto en el fondo ajedrezado como en el círculo central). Esto representa gráficamente los bits que se degradaron (efecto bit-flip hacia cero) debido al tiempo de decaimiento y la probabilidad de apagado de los condensadores de la memoria.

5. Limitaciones del Estudio

- El simulador se ejecuta íntegramente en un entorno de máquina virtual Linux, sin acceso a una Raspberry Pi ni a ningún otro dispositivo IoT. Por tanto, los datos no provienen de un volcado real de memoria SDRAM, sino de una imagen sintética almacenada en un buffer en memoria RAM del sistema anfitrión.
- Aunque el modelo de probabilidad de apagado se ha calibrado a partir de datos empíricos del artículo de Won et al., no contempla efectos secundarios como la descorrelación de bits adyacentes, la influencia de la humedad o las variaciones propias de cada fabricante de chips. Se asume un comportamiento homogéneo para toda la memoria.
- El simulador solo trabaja con imágenes en escala de grises de tamaño fijo (por defecto 256×256) y formato PGM. No se contemplan otros tipos de datos (texto, binarios, claves criptográficas) que podrían ser el objetivo real de un atacante.
- El uso de números pseudoaleatorios hace que los resultados varíen ligeramente entre ejecuciones. Aunque la tasa de recuperación se mantiene estable, el patrón de bits erróneos es diferente cada vez, lo que introduce una fuente de variabilidad no presente en un experimento físico controlado.
- El simulador no considera el tiempo necesario para leer físicamente la memoria y transferir los datos a un medio externo (por ejemplo, por UART o USB), que en un ataque real podría limitar la ventana de oportunidad.

6. Conclusiones y Trabajo Futuro

En este trabajo se ha desarrollado un simulador computacional que reproduce el comportamiento de un ataque de arranque en frío sobre una memoria SDRAM, basado en el modelo experimental propuesto por Won et al. para dispositivos IoT. El simulador, implementado en C, genera una imagen de prueba, aplica un modelo de degradación de bits dependiente de la temperatura y el tiempo de decaimiento, y calcula la tasa de recuperación. Los resultados obtenidos muestran que a $-30\text{ }^{\circ}\text{C}$ la recuperación supera el 99.9 % incluso tras 5 segundos, mientras que a temperatura ambiente cae por debajo del 60 %.

7. Referencias

- [1] Y.-S. Won, J.-Y. Park, D.-G. Han, and S. Bhasin, “Practical Cold boot attack on IoT device - Case study on Raspberry Pi -,” Temasek Laboratories, Nanyang Technological University, Singapore, [En línea]. Disponible en: <https://dr.ntu.edu.sg/handle/10356/143500>.
- [2] J. A. Halderman et al., “Lest We Remember: Cold Boot Attacks on Encryption Keys,” in *Proceedings of the 17th USENIX Security Symposium*, Jul. 2008, pp. 45–60, [En línea]. Disponible en: https://www.usenix.org/legacy/event/sec08/tech/full_papers/halderman/halderman.pdf.
- [3] S. F. Yitbarek, M. T. Aga, R. Das, and T. Austin, “Cold Boot Attacks are Still Hot: Security Analysis of Memory Scramblers in Modern Processors,” in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2017, pp. 313–324, [En línea]. Disponible en: <https://www.semanticscholar.org/paper/Cold-Boot-Attacks-are-Still-Hot%3A-Security-Analysis-Yitbarek-Aga/7a4b8c9e1f2d3a4b5c6d7e8f9a0b1c2d3e4f5a6b>.

- [4] J. Bauer, M. Gruhn, and F. C. Freiling, “Lest we forget: Cold-boot attacks on scrambled DDR3 memory,” *Digital Investigation*, vol. 16, pp. S65–S74, 2016, [En línea]. Disponible en: <https://cris.fau.de/converis/publication/218328270>.
- [5] M. R. Albrecht, A. Deo, and K. G. Paterson, “Cold Boot Attacks on Ring and Module LWE Keys Under the NTT,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 3, pp. 173–213, Aug. 2018, [En línea]. Disponible en: <https://eprint.iacr.org/2018/672>.
- [6] Broadcom, “BCM2835 ARM Peripherals,” Raspberry Pi Documentation, [En línea]. Disponible en: <https://www.raspberrypi.org/documentation/hardware/raspberrypi/bcm2835/BCM2835-ARM-Peripherals.pdf>.
- [7] anfractuosity, “ramrecovery: Simple demo illustrating remanence of data in RAM (see Cold boot attack) using a Raspberry Pi,” GitHub repository, [En línea]. Disponible en: <https://github.com/anfractuosity/ramrecovery>.